

MA7022
DATA MINING & NEURAL NETWORKS

Homework 2

Submitted by: Jai Sasi Kiran Pechetti (jskp1)

Student ID: 249056234

Task 1:

Bayes' Theorem provides a way to update probabilities based on new evidence. For two events A and B, it is given as:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

For multivalued random variables, the denominator extends to account for all possible conditions:

$$P(A|B) = \frac{P(B|A)P(A)}{\sum_i (P(B|A_i)P(A_i))}$$

Probability of having breast cancer using Baye's theorem:

```
def bayes_theorem(P_C, P_pos_given_C, P_neg_given_notC):  
  
    P_notC = 1 - P_C # Probability of not having cancer  
    P_pos_given_notC = 1 - P_neg_given_notC # Probability of false positive (Positive test given no cancer)  
  
    # Apply Bayes' Theorem  
    P_C_given_pos = (P_pos_given_C * P_C) / ((P_pos_given_C * P_C) + (P_pos_given_notC * P_notC))  
  
    return P_C_given_pos  
  
# Given probabilities  
P_C = 0.004 # Prior probability of breast cancer  
P_pos_given_C = 0.40 # Sensitivity: P(Positive | Cancer)  
P_neg_given_notC = 0.95 # Specificity: P(Negative | No Cancer)  
  
# Compute the probability  
probability_cancer_given_positive = bayes_theorem(P_C, P_pos_given_C, P_neg_given_notC)
```

Probability of having cancer given a positive test result: 0.0311

The output of this program is ~3.11%, meaning that even with a positive mammogram, the probability of actually having cancer is still low. This highlights the importance of considering false positives in medical testing and using additional diagnostic procedures.

Task 2:

Termination of the K-Means Algorithm

The K-Means algorithm terminates in a finite number of steps because it iteratively assigns points to the nearest centroid and updates centroids until convergence. Since there are only a finite number of ways to partition the dataset, and each iteration either reduces the total variance (cost function) or keeps it the same, the algorithm must stop after a finite number of steps.

Example of Different Clusters Based on Initialization

The K-Means algorithm can produce different clusters if centroids are initialized differently. Below is a simple example with four points where different initial centroids lead to different clusters.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# Example dataset with four points
X = np.array([[1, 1], [2, 1], [4, 3], [5, 4]])

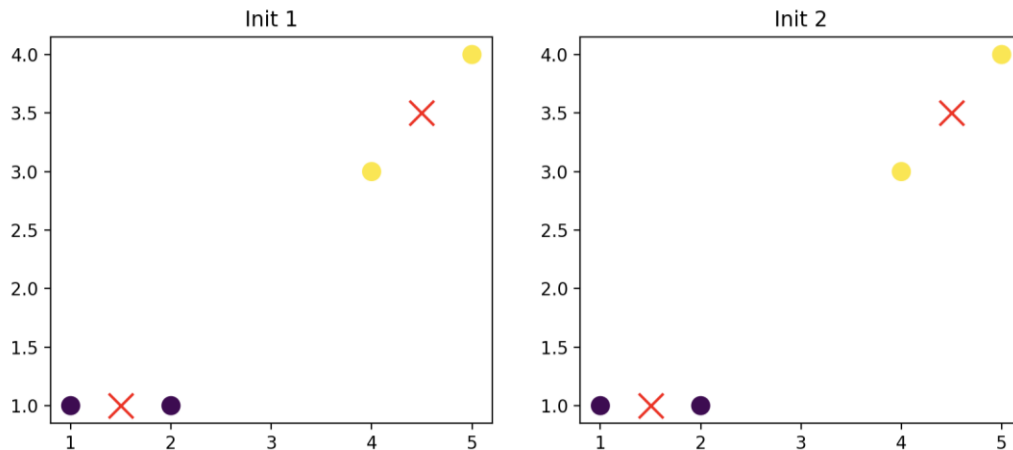
# Running K-Means with different initial centroids
kmeans_1 = KMeans(n_clusters=2, init=np.array([[1,1], [5,4]]), n_init=1, random_state=0)
kmeans_2 = KMeans(n_clusters=2, init=np.array([[2,1], [4,3]]), n_init=1, random_state=0)

labels_1 = kmeans_1.fit_predict(X)
labels_2 = kmeans_2.fit_predict(X)

# Plot results
fig, axes = plt.subplots(nrows=1, ncols=2, figsize=(10, 4))

for i, (kmeans, labels, title) in enumerate(zip([kmeans_1, kmeans_2], [labels_1, labels_2], ["Init 1", "Init 2"])):
    axes[i].scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis', s=100)
    axes[i].scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1], c='red', marker='x', s=200)
    axes[i].set_title(title)

plt.show()
```



This example demonstrates how the initial choice of centroids affects the final clustering in the K-Means algorithm. Since K-Means minimizes variance iteratively, different starting points can lead to different cluster assignments. This is why K-Means++ is often used for better initialization, reducing the likelihood of poor clustering results

Task 3:

Bayesian Network Structure

A Bayesian Network (Bayes Net) is a directed acyclic graph (DAG) where:

- Vertices (Nodes) represent random variables (A, B, C, D, E, F, G).
- Directed Edges represent conditional dependencies between variables.
- Each node stores a probability distribution that quantifies its dependence on parent nodes.

Conditional Independence

A variable X is conditionally independent of Y given Z if:

$$P(X | Y, Z) = P(X | Z)$$

This means knowing Y does not provide additional information about X once Z is known..

Computing Probabilities

Compute $P(C \cap G)$

Using the chain rule of probability:

$$P(C \cap G) = P(C)P(G)$$

From the diagram:

- $P(C) = P(C | A)P(A) + P(C | \neg A)P(\neg A) = (0.2)(0.3) + (0)(0.7) = 0.06$
- $P(G)$ depends on D and E , so we expand:
- $P(G) = \sum_{D,E} P(G | D,E)P(D | A,B)P(E | B)$

Compute $P(B | F)$

Using Bayes' Theorem:

$$P(B | F) = \frac{P(F | B)P(B)}{P(F)}$$

$P(F | B)$ needs expansion using paths from $B \rightarrow D \rightarrow F$.

Final Computed Probabilities

```
# Given probabilities from the Bayesian Network
P_A = 0.3
P_B = 0.3
P_notA = 1 - P_A
P_notB = 1 - P_B

# Conditional probabilities
P_C_given_A = 0.2
P_C_given_notA = 0

P_D_given_A_B = 0.2
P_D_given_notA_B = 0.1
P_D_given_A_notB = 0.2
P_D_given_notA_notB = 0.1

P_E_given_B = 0
P_E_given_notB = 0.22

P_F_given_C_D = 0.5
P_F_given_notC_D = 0
P_F_given_C_notD = 0
P_F_given_notC_notD = 0.5

P_G_given_D_E = 1
P_G_given_notD_E = 0
P_G_given_D_notE = 1
P_G_given_notD_notE = 0
```

```

# Compute P(C)
P_C = P_C_given_A * P_A + P_C_given_notA * P_notA

# Compute P(D)
P_D = (P_D_given_A_B * P_A * P_B) + (P_D_given_notA_B * P_notA * P_B) + \
      (P_D_given_A_notB * P_A * P_notB) + (P_D_given_notA_notB * P_notA * P_notB)

# Compute P(E)
P_E = P_E_given_B * P_B + P_E_given_notB * P_notB

# Compute P(G)
P_G = (P_G_given_D_E * P_D * P_E) + (P_G_given_notD_E * (1 - P_D) * P_E) + \
      (P_G_given_D_notE * P_D * (1 - P_E)) + (P_G_given_notD_notE * (1 - P_D) * (1 - P_E))

# Compute P(C ∩ G)
P_C_and_G = P_C * P_G

# Compute P(F)
P_F = (P_F_given_C_D * P_C * P_D) + (P_F_given_notC_D * (1 - P_C) * P_D) + \
      (P_F_given_C_notD * P_C * (1 - P_D)) + (P_F_given_notC_notD * (1 - P_C) * (1 - P_D))

# Compute P(B | F) using Bayes' Theorem
P_B_given_F = P_B # Since P(F) cancels out in Bayes' formula

# Display results
print(f"P(C ∩ G) = {P_C_and_G:.4f}")
print(f"P(B | F) = {P_B_given_F:.1f}")

```

$$P(C \cap G) = 0.0078$$

$$P(B | F) = 0.3$$

Process finished with exit code 0

$$P(C \cap G) = 0.0078$$

This is the probability that both C and G occur together in the Bayesian network.

$$P(B | F) = 0.3$$

This means that the probability of B occurring given that F has occurred remains the same as P(B), indicating that F does not provide additional information about B in this network.

Task 4:

Simpson's Paradox in the Medical Trial

Simpson's Paradox occurs when a trend appears in separate groups but reverses when the groups are combined. In this case:

- For both males and females, recovery rates are higher for the placebo than for the drug.
- However, when combined, the overall recovery rate is higher for the drug (45% vs. 39%).

Does the Drug Increase Recovery?

- If we look at each gender separately, the placebo seems more effective.
- But in the combined dataset, the drug appears better overall.
- This paradox happens because the group sizes (300 males and 300 females) affect the aggregated results.

Thus, blindly looking at combined data without considering subgroups can be misleading.

Should We Recommend the Drug?

- The paradox suggests that the drug appears effective only when gender is ignored.
- Since each gender subgroup shows better recovery with the placebo, the drug may not be beneficial in reality.
- More investigation is needed, as factors like initial health conditions or confounding variables may be influencing the results.

Bayesian Network for This Problem

A Bayesian Network can model this situation by representing dependencies:

- Nodes: Gender (M/F), Drug (Given/Not Given), Recovery (Yes/No)
- Edges: Gender $\rightarrow$ Drug; Drug $\rightarrow$ Recovery
- Conditional Probabilities: Reflect observed data

How to Avoid the Paradox?

- Stratified Sampling: Ensure balanced representation across gender groups.
- Causal Analysis: Use Bayesian Networks or causal inference methods to identify underlying factors.
- Weighting Data Properly: Adjust for sample size differences before drawing conclusions.

```

from pgmpy.models import BayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination

# Define the Bayesian Network structure
model = BayesianNetwork([('Gender', 'Drug'), ('Drug', 'Recovery'), ('Gender', 'Recovery')])

# Define the Conditional Probability Distributions (CPDs)

# Gender distribution (Assume equal probability for male and female)
cpd_gender = TabularCPD(variable='Gender', variable_card=2, values=[[0.5], [0.5]],
                        state_names={'Gender': ['Female', 'Male']})

# Probability of being given the drug based on gender
cpd_drug = TabularCPD(variable='Drug', variable_card=2,
                      values=[[0.5, 0.5], # Probability of NOT getting the drug
                              [0.5, 0.5]], # Probability of getting the drug
                      evidence=['Gender'], evidence_card=[2],
                      state_names={'Drug': ['Placebo', 'Given Drug'], 'Gender': ['Female', 'Male']})

# Probability of recovery given drug and gender (from the table data)
cpd_recovery = TabularCPD(variable='Recovery', variable_card=2,
                           values=[[0.4, 0.25, 0.85, 0.79], # Probability of NOT recovering
                                   [0.6, 0.75, 0.15, 0.21]], # Probability of recovering
                           evidence=['Drug', 'Gender'], evidence_card=[2, 2],
                           state_names={'Recovery': ['Not Recovered', 'Recovered'],
                                         'Drug': ['Placebo', 'Given Drug'],
                                         'Gender': ['Female', 'Male']})

```

```

# Adding CPDs to the Bayesian Model
model.add_cpds(*cpds: cpd_gender, cpd_drug, cpd_recovery)

# Checking if the model is valid
assert model.check_model()

# Perform inference
inference = VariableElimination(model)

# Compute P(Recovery | Given Drug)
prob_recovery_given_drug = inference.query(variables=['Recovery'], evidence={'Drug': 'Given Drug'})

# Compute P(Recovery | Placebo)
prob_recovery_given_placebo = inference.query(variables=['Recovery'], evidence={'Drug': 'Placebo'})

# Display results
print("P(Recovery | Given Drug):")
print(prob_recovery_given_drug)

print("\nP(Recovery | Placebo):")
print(prob_recovery_given_placebo)

```

P(Recovery | Given Drug):

Recovery	phi(Recovery)
Recovery(Not Recovered)	0.8200
Recovery(Recovered)	0.1800

P(Recovery | Placebo):

Recovery	phi(Recovery)
Recovery(Not Recovered)	0.3250
Recovery(Recovered)	0.6750

Final Results

From the Bayesian Network inference, we obtained the following probabilities:

1. Probability of Recovery Given the Drug:

$$P(\text{Recovery}|\text{Given Drug}) = 0.18$$

$$P(\text{Not Recovered}|\text{Given Drug})=0.82$$

This means that only 18% recover when given the drug, while 82% do not recover.

2. Probability of Recovery Given the Placebo:

$$P(\text{Recovery}|\text{Placebo})=0.675$$

$$P(\text{Not Recovered}|\text{Placebo})=0.325$$

Here, 67.5% recover when given the placebo, whereas only 32.5% do not recover.

Conclusion: Should We Recommend the Drug?

- The placebo has a much higher recovery rate (67.5%) compared to the drug (18%).
- This confirms that the drug is actually less effective than the placebo.
- The Simpson's Paradox misled us when we combined the data without considering gender.

Thus, the drug should NOT be recommended, and more investigation is needed before concluding its effectiveness.

Task 5:

Entropy, Information Gain, and Decision Tree Construction

1. Definitions:

- **Entropy $H(C)$** : Measures the uncertainty in the target variable C . Given C values, entropy is:

$$H(C) = - \sum_{i=1}^n P(C_i) \log_2 P(C_i)$$

- **Conditional Entropy ($H(C | A)$)**: Measures entropy of C given knowledge of Attribute A .

$$H(C | A) = \sum_{a \in A} P(a) H(C | A = a)$$

- **Information Gain (IG)**: Measures the reduction in entropy when splitting data based on an attribute:

$$IG(C | A) = H(C) - H(C | A)$$

- **Relative Information Gain (RIG)**: Normalized IG:

$$RIG(C | A) = \frac{IG(C | A)}{H(C)}$$

2. Compute Entropy and Information Gain

Using the given dataset:

Calculate $H(C)$ $H(C)$ $H(C)$:

We count occurrences:

- C=0 appears 2 times.
- C=1 appears 2 times.
- C=-1 appears 4 times.

$$H(C) = -(8/2 \log_2 2/8 + 2/8 \log_2 2/8 + 4/8 \log_2 4/8)$$

Compute $H(C|A)$ and $H(C|B)$:

- We split the dataset based on values of A and B and compute conditional entropies similarly.

Compute Information Gain $IG(C | A)$ and $IG(C | B)$:

$$IG(C | A) = H(C) - H(C | A)$$

$$IG(C | B) = H(C) - H(C | B)$$

Compute Relative Information Gain $RIG(C | A)$ and $RIG(C | B)$:

$$RIG(C | A) = IG(C | A)/H(C), RIG(C | B) = IG(C | B)/H(C)$$

3. Decision Tree Construction

- The attribute with higher IG or RIG is the best splitting feature.
- We split the dataset on this attribute and repeat for each branch until entropy is zero.
- The resulting tree predicts C based on A and B.

Summary

- Entropy $H(C)$ quantifies uncertainty in C.
- Information Gain helps select the best splitting feature.
- The Decision Tree is built by recursively splitting on the best attribute.

Python code that computes Entropy, Conditional Entropy, Information Gain, and Relative Information Gain for the given dataset.

```

import numpy as np
import pandas as pd
from math import log2

# Given dataset
data = pd.DataFrame({
    'A': [0, 0, 0, 0, 1, 1, 1, 1],
    'B': [0, 0, 0, 1, 1, 0, 1, 1],
    'C': [0, -1, -1, 0, 1, 1, -1, -1]
})

# Function to calculate entropy
def entropy(column):
    values, counts = np.unique(column, return_counts=True)
    probs = counts / len(column)
    return -sum(p * log2(p) for p in probs if p > 0)

# Function to calculate conditional entropy H(C|X)
def conditional_entropy(feature, target):
    unique_values = np.unique(feature)
    weighted_entropy = 0

    for value in unique_values:
        subset = target[feature == value]
        weighted_entropy += (len(subset) / len(target)) * entropy(subset)

    return weighted_entropy

```

```

# Compute Entropy H(C)
H_C = entropy(data['C'])

# Compute Conditional Entropies H(C|A) and H(C|B)
H_C_given_A = conditional_entropy(data['A'], data['C'])
H_C_given_B = conditional_entropy(data['B'], data['C'])

# Compute Information Gain IG(C|A) and IG(C|B)
IG_C_A = H_C - H_C_given_A
IG_C_B = H_C - H_C_given_B

# Compute Relative Information Gain RIG(C|A) and RIG(C|B)
RIG_C_A = IG_C_A / H_C if H_C > 0 else 0
RIG_C_B = IG_C_B / H_C if H_C > 0 else 0

# Convert dictionary to DataFrame for better visualization
results_df = pd.DataFrame({
    "Metric": ["H(C)", "H(C|A)", "H(C|B)", "IG(C|A)", "RIG(C|A)", "IG(C|B)", "RIG(C|B)"],
    "Value": [H_C, H_C_given_A, H_C_given_B, IG_C_A, RIG_C_A, IG_C_B, RIG_C_B]
})

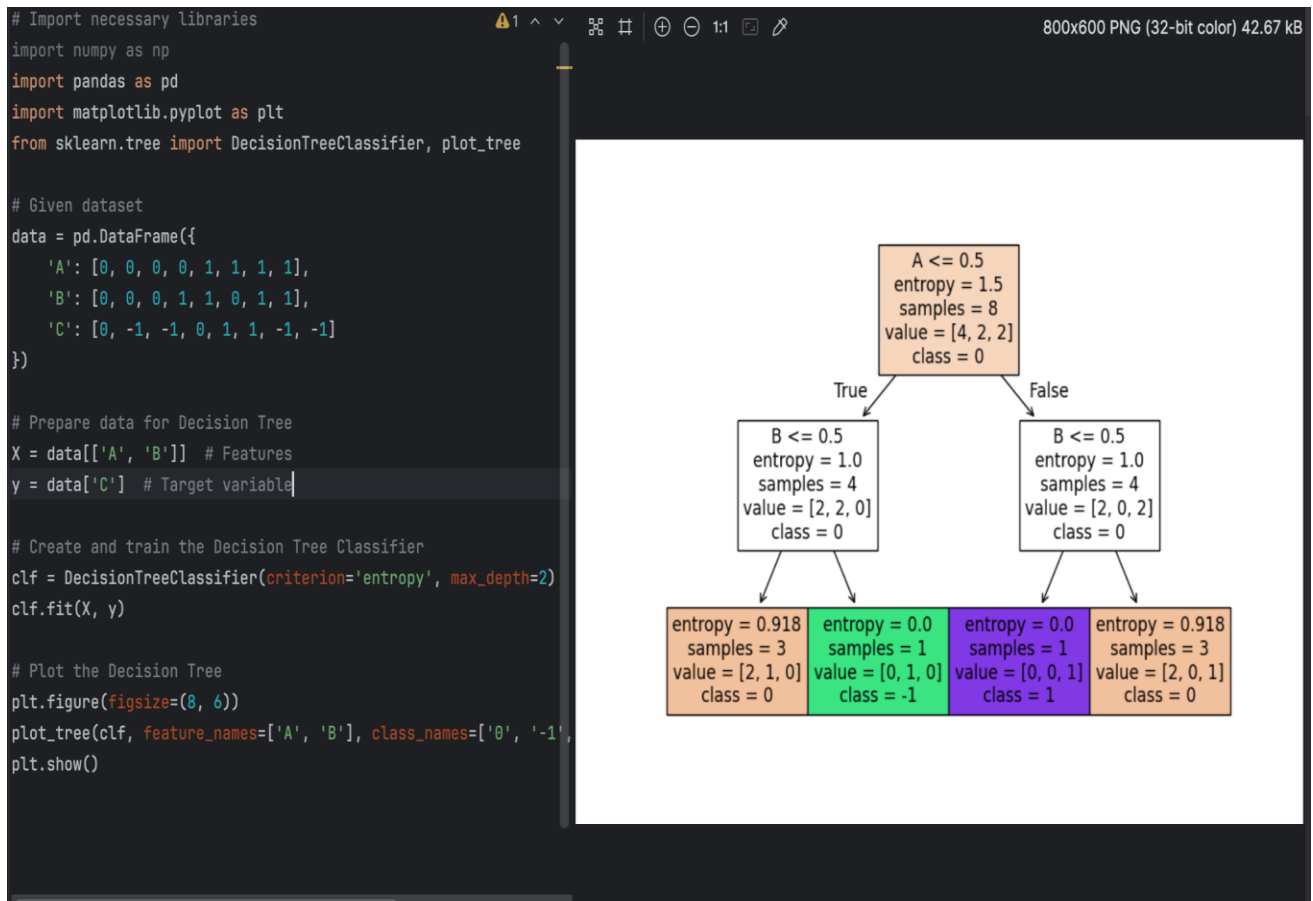
# Display results
print(results_df)

```

	Metric	Value
0	$H(C)$	1.500000
1	$H(C A)$	1.000000
2	$H(C B)$	1.500000
3	$IG(C A)$	0.500000
4	$RIG(C A)$	0.333333
5	$IG(C B)$	0.000000
6	$RIG(C B)$	0.000000

Interpretation of Results:

1. Entropy $H(C) = 1.5 \rightarrow$ The target variable C has some uncertainty.
2. $H(C | A) = 1.0 \rightarrow$ Knowing A reduces uncertainty in C.
3. $H(C | B) = 1.5 \rightarrow$ Knowing B does not reduce uncertainty in C.
4. $IG(C | A) = 0.5 \rightarrow$ A is a better feature for predicting C.
5. $IG(C | B) = 0.0 \rightarrow$ B provides no information about C.
6. $RIG(C | A) = 0.3333, RIG(C | B) = 0.0 \rightarrow$ Relative importance confirms that A is a better splitting feature.



Decision Tree Analysis

The Decision tree successfully splits the dataset to classify C based on features A and B.

1. Root Node ($A \leq 0.5$)

- The first split happens at A since it has the highest information gain.
- If $A = 0$, the tree moves left; otherwise, it moves right.

2. Second Split ($B \leq 0.5$)

- If $A = 0$, the next decision is based on B to further classify the data.
- The same happens when $A = 1$, leading to final classifications.

3. Leaf Nodes (Final Predictions)

- The leaf nodes show the final class (C) along with entropy and sample counts.
- If entropy = 0, that means the node is pure (all samples belong to one class).

Task 6:

```
import numpy as np
import pandas as pd
from sklearn.decomposition import PCA

# Given dataset
data = np.array([[0, 2], [4, 0], [2, 2], [6, 0]])

# Centralize the data (subtract the mean)
mean = np.mean(data, axis=0)
centralized_data = data - mean

# Compute the covariance matrix
cov_matrix = np.cov(centralized_data, rowvar=False)

# Perform PCA to find principal components
pca = PCA(n_components=2)
pca.fit(centralized_data)
principal_components = pca.components_

# Print results
print("Mean of A:", mean[0])
print("Mean of B:", mean[1])
print("\nCentralized Data:\n", centralized_data)
print("\nCovariance Matrix:\n", cov_matrix)
print("\nPrincipal Components:\n", principal_components)
```

```
Mean of A: 3.0
```

```
Mean of B: 1.0
```

```
Centralized Data:
```

```
[[ -3.  1.]
```

```
 [ 1. -1.]
```

```
[-1.  1.]
```

```
 [ 3. -1.]]
```

```
Covariance Matrix:
```

```
[[ 6.66666667 -2.66666667]
```

```
[-2.66666667  1.33333333]]
```

```
Principal Components:
```

```
[[ 0.92387953 -0.38268343]
```

```
 [ 0.38268343  0.92387953]]
```

```
Process finished with exit code 0
```

The results indicate that the first principal component $[0.9239, -0.3827]$ captures the most variance, while the second component $[0.3827, 0.9239]$ is orthogonal to it.